Worksheet 13 - Cloud & Container Security (4 hrs)

Part 1 - Student Information

Name	Student ID	Date	Group
Danaiphong Songsikhwa	6631503106	7 November 2026	Not provided

AI use: I used Claude to help me run the lab commands, save the output into the `evidence/` folder and draft some answers. I checked the numbers in this worksheet with the saved output. In the Audit the AI part, the AI answer is a typical "quick fix" answer that I wanted to test.

Branch: `wk13_6631503106` (separate branch, not merged into `main`).

Lab environment: Docker Engine 29.9.3, Trivy 0.74.0 (`aquasec/trivy:latest`), Linux.

Part 2 - Lecture Questions

Answer in your own words (2-4 sentences each).

1. Explain the **shared-responsibility model**. In `harden.md` it says "the cloud provider secures *of* the cloud; you secure what you put *in* it." Give one concrete example of each side for an S3-backed app.

Answer: Security in the cloud is split between the provider and the customer. AWS takes care of the data centers, the servers and the S3 service itself, for example other customers cannot read the disks where my files are stored. My side is the configuration. For an app that uses S3, I have to keep the bucket private, give the app's IAM role only the permissions it needs, and not put access keys inside the container image.

2. `iam-policy-insecure.json` uses `"Action": "*"` and `"Resource": "*"`. Describe **why this violates least privilege** and map it to CWE-732 vs CWE-269 - what is the difference between the two CWEs here?

Answer: Least privilege means giving only the actions that are needed, only on the resources that are needed. Our app just reads files from one bucket, but this policy allows every action on everything in the account, so it is basically an admin account. I map `"Action": "*"` to CWE-269 (improper privilege management) because the identity can do things it should never do, like `iam:CreateUser`. I map `"Resource": "*"` to CWE-732 (incorrect permission assignment) because even a normal action like reading a file works on every bucket, not only `lab-app-bucket/app/*`. In short, CWE-269 is about what you can do and CWE-732 is about where you can do it.

3. The insecure Dockerfile bakes `ENV API_TOKEN=sk_live_...`. Explain how an attacker recovers that secret from a shipped image (name the `docker` commands) and why this is **CWE-798 / CWE-200**.

Answer: The attacker only needs to pull the image. Then `docker history --no-trunc week13-insecure:lab` shows the full command of each layer, including `ENV API_TOKEN=sk_live_supersecret_DO_NOT_DO_THIS`, and `docker inspect --format '{{json .Config.Env}}' week13-insecure:lab` shows it in the config. I tried both in Task 3 and got the token and the AWS key. It is CWE-798 because the credential is hard-coded in the build, and CWE-200 because it is exposed to anyone who has the image. If the line is removed later, the old image tags in the registry still have the key, so it has to be rotated too.

4. Why is `FROM python:latest` (CWE-1104/CWE-16) a security and reproducibility problem, and how does pinning to an `@sha256:` digest fix it?

Answer: `latest` can point to a different image at any time. When I built it today I got Python 3.15.2 on Debian 13, which nobody tested with this app. If the upstream image is broken or compromised, the build will take it without any warning. A digest like `@sha256:cdbd05fb...` always means the same image, so the build is repeatable and a change of base image has to be done on purpose. The problem is that the old CVEs are also frozen, so the digest should be updated regularly (Dependabot or Renovate can do this) and scanned again.

5. What does a **distroless** base image remove (relative to `python:3.11-slim`), and how does that shrink the attack surface for an RCE or container-escape attacker?

Answer: A distroless image has the Python runtime and its libraries, but no shell, no package manager like `apt`, and almost no other tools such as `curl`. I tested `docker run --entrypoint sh week13-hardened:lab -c id` and it failed with `exec: "sh": executable file not found`. So if an attacker gets code execution in the app, it is much harder to open a shell, install tools or download more malware. It also has fewer packages that can have CVEs. The image size was 125 MB, compared to 1.65 GB for the insecure image.

Part 3 - Hands-on Lab (180 min)

Learning goals: find cloud/container misconfigurations with Trivy, map each to a CWE, and rebuild the IAM policy + Dockerfile to least privilege / hardened so the findings drop.

Prerequisites: Docker installed and running; the `week13-cloud-container/` lab folder; internet access to pull `aquasec/trivy:latest`.

Environment setup (real commands)

```
cd labs/week13-cloud-container

# Option A - the provided wrapper (trivy config over the Dockerfiles; IAM JSON is reviewed manually, see
 Task 2):
bash scan.sh

# Option B - run Trivy directly (MEDIUM is in the filter on purpose: one of the
# three rules Task 1 asks you to cite is rated MEDIUM, and HIGH,CRITICAL hides it):
docker run --rm -v "$PWD:/src" aquasec/trivy:latest config --severity MEDIUM,HIGH,CRITICAL /src

# (Optional) build + CVE-scan the hardened image:
docker build -f Dockerfile.hardened -t week13-hardened:lab .

docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --severity
HIGH,CRITICAL week13-hardened:lab
```

What to submit per task: the command(s) you ran + the relevant output (finding count or the specific finding line), a screenshot of the Trivy/terminal result, and a 2-3 sentence mitigation explaining *why* the fix reduces risk (cite the CWE).

Tasks

- **Task 0 - Onboarding (15 min).** Confirm Docker works (`docker version`), run `bash scan.sh`, and record the **baseline finding count** against `Dockerfile.insecure` (Trivy's `config` scanner does not parse standalone IAM policy JSON, so `iam-policy-insecure.json` is reviewed manually instead - see Task 2). **Deliverable:** screenshot of the baseline scan.

Task 0 - Answer

Commands:

```
docker version --format '{{.Server.Version}}'    # 29.9.3
bash scan.sh
```

Trivy report summary (full output in `evidence/scan-before.txt`):

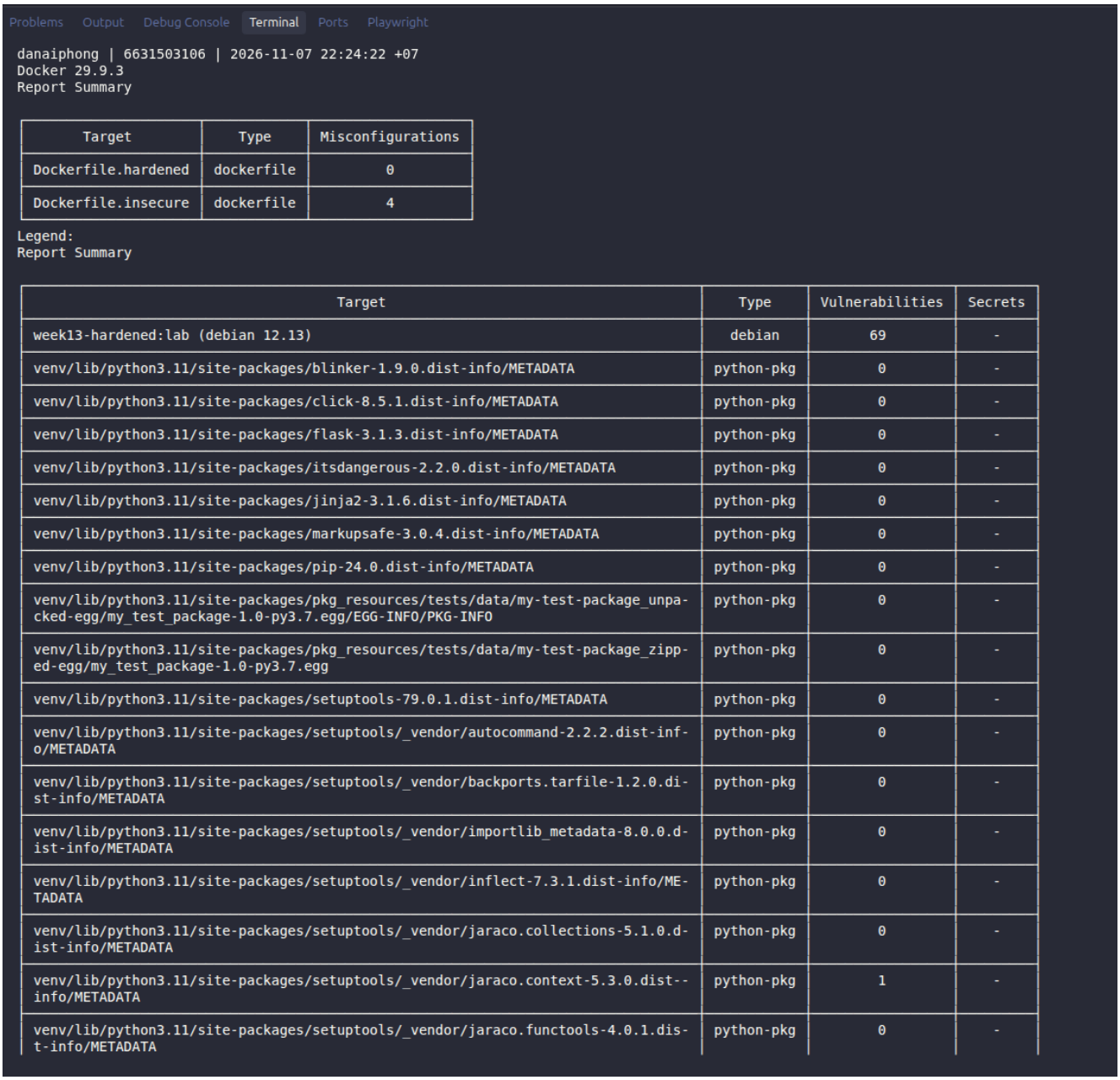
```
┌──────────────────────┬────────────┬───────────────────┐
│        Target        │    Type    │ Misconfigurations │
├──────────────────────┼────────────┼───────────────────┤
│ Dockerfile.hardened  │ dockerfile │         0         │
```

```
┌──────────────────────┬────────────┬─────────────────┐
│ Dockerfile.insecure  │ dockerfile │        4        │
└──────────────────────┴────────────┴─────────────────┘
Dockerfile.insecure (dockerfile)
Tests: 25 (SUCCESSES: 21, FAILURES: 4)
Failures: 4 (MEDIUM: 1, HIGH: 1, CRITICAL: 2)
```

Baseline: `Dockerfile.insecure` has 4 findings (1 MEDIUM, 1 HIGH, 2 CRITICAL) and `Dockerfile.hardened` has 0. The IAM JSON files are not in this table because Trivy does not scan them, so I reviewed them myself in Task 2.

Task 0 screenshot (the second table is the optional `trivy image` part of `scan.sh`. It shows the 69 OS CVEs of the hardened image that I talk about in Task 4.)

Task 0 - docker version and baseline scan.sh summary (insecure 4, hardened 0)



Problems Output Debug Console Terminal Ports Playwright

```
danaiphong | 6631503106 | 2026-11-07 22:24:22 +07
Docker 29.9.3
Report Summary
```

Target	Type	Misconfigurations
Dockerfile.hardened	dockerfile	0
Dockerfile.insecure	dockerfile	4

```
Legend:
Report Summary
```

Target	Type	Vulnerabilities	Secrets
week13-hardened:lab (debian 12.13)	debian	69	-
venv/lib/python3.11/site-packages/blinker-1.9.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/click-8.5.1.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/flask-3.1.3.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/itsdangerous-2.2.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/jinja2-3.1.6.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/markupsafe-3.0.4.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/pip-24.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/pkg_resources/tests/data/my-test-package_unpacked-egg/my_test_package-1.0-py3.7.egg/EGG-INFO/PKG-INFO	python-pkg	0	-
venv/lib/python3.11/site-packages/pkg_resources/tests/data/my-test-package_zipped-egg/my_test_package-1.0-py3.7.egg	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools-79.0.1.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/autocommand-2.2.2.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/backports.tarfile-1.2.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/importlib_metadata-8.0.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/inflect-7.3.1.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/jaraco.collections-5.1.0.dist-info/METADATA	python-pkg	0	-
venv/lib/python3.11/site-packages/setuptools/_vendor/jaraco.context-5.3.0.dist-info/METADATA	python-pkg	1	-
venv/lib/python3.11/site-packages/setuptools/_vendor/jaraco.functools-4.0.1.dist-info/METADATA	python-pkg	0	-

- **Task 1 - Misconfig Hunt: container image (40 min).** *Goal:* find every defect in `Dockerfile.insecure`. *Steps:* read each `# DEFECT:` comment, then locate the matching Trivy finding where one exists - only 3 of the 6 (unpinned `:latest`, runs as root, secrets in ENV) map to a Trivy config rule (`DS-0001`, `DS-0002`, `DS-0031`); `COPY . .`, `chmod -R 777`, and unpinned `pip install` are instruction-level RUN/COPY semantics Trivy's Dockerfile scanner doesn't check, so identify those 3 by code review instead. Build a table: defect → CWE → Trivy rule/severity (write

"manual review - no Trivy rule" where none exists). *Deliverable:* completed table (6 rows) using the IDs from `harden.md`.

Task 1 - Answer

# (harden.md)	Defect in `Dockerfile.insecure`	CWE	Trivy rule / severity
1	Line 11 `FROM python:latest` (unpinned tag)	CWE-1104 / CWE-16	DS-0001 / MEDIUM
2	Lines 16-17 `ENV API_TOKEN=...` and `ENV AWS_SECRET_ACCESS_KEY=...`	CWE-798 / CWE-200	DS-0031 / CRITICAL (2 findings, one for each variable)
3	Line 24 `COPY . .` (copies the whole folder)	CWE-538	manual review, no Trivy rule
4	Line 31 `RUN chmod -R 777 /app`	CWE-732	manual review, no Trivy rule
5	No `USER` line, so it runs as root	CWE-250	DS-0002 / HIGH (no line number because the line is missing)
6	Line 27 `RUN pip install -r requirements.txt` (`Flask>=3.1.3`, no exact version, no hashes)	CWE-1104	manual review, no Trivy rule

The 4 Trivy findings only cover 3 of the 6 defects, because DS-0031 appears two times. To check the manual ones, I ran the image (`evidence/insecure-runtime.txt`):

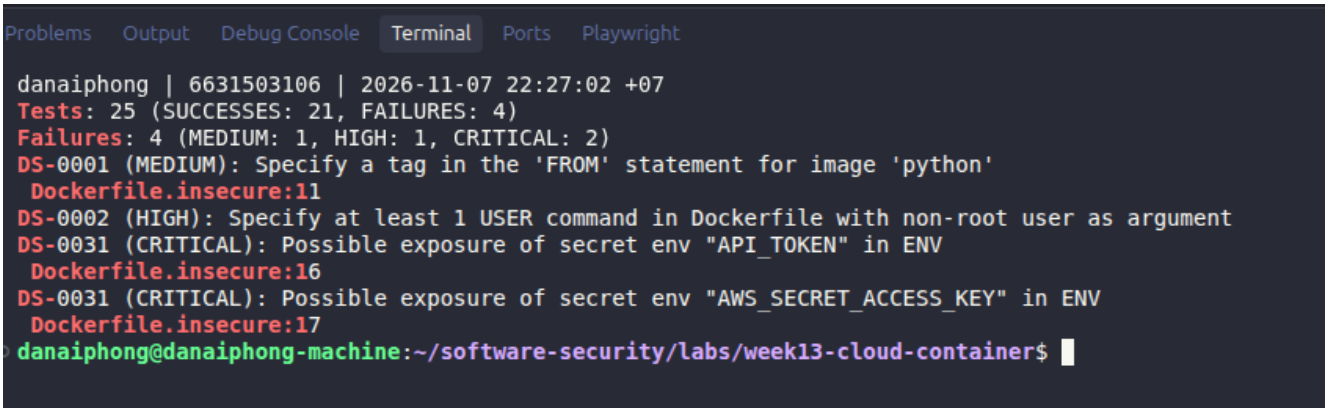
```
$ docker run --rm week13-insecure:lab sh -c 'id; ls -ld /app; ls -la /app'
uid=0(root) gid=0(root) groups=0(root)
drwxrwxrwx 1 root root 4096 Nov  7 15:01 /app
-rwxrwxrwx 1 root root  1535 Aug 15 15:35 Dockerfile.insecure
-rwxrwxrwx 1 root root   638 Aug 15 15:35 app.py
drwxrwxrwx 1 root root  4096 Nov  7 15:01 evidence
-rwxrwxrwx 1 root root 10029 Aug 15 15:35 worksheet.md
...
```

The process is root (defect 5) and `/app` is `drwxrwxrwx` (defect 4). Because of `COPY . .`, the Dockerfiles, the worksheet and even my `evidence/` folder ended up inside the image (defect 3). For defect 6, `Flask>=3.1.3` gave me Flask 3.1.3 and Werkzeug 3.1.9 today, but the same line could install other versions next month.

Mitigation: pin the base image by digest (CWE-1104), keep secrets out of the image (CWE-798), copy only `app.py` (CWE-538), use normal permissions instead of 777 (CWE-732), run as `USER 65532:65532` (CWE-250), and pin requirements with hashes (CWE-1104). Each of these closes one way for an attacker or a bad upstream update to get into the container.

Task 1 screenshot:

Task 1 - Trivy findings DS-0001, DS-0002 and DS-0031 for Dockerfile.insecure



• **Task 2 - Misconfig Hunt: IAM (30 min).** *Goal:* document why `iam-policy-insecure.json` is full-admin. *Steps:* identify the `Action:*` / `Resource:*` / no-`Condition` problems; map to CWE-269 / CWE-732. *Deliverable:* 3-row table + one sentence on the blast radius if these creds leak.

Task 2 - Answer

Manual review of `iam-policy-insecure.json` (Trivy does not scan this file):

# (harden.md)	Problem	CWE	Why it is dangerous
7	`"Action": "*"` (line 8)	CWE-269	Every API call is allowed, including `iam:*`, so the user can make new admin users or new keys.
8	`"Resource": "*"` (line 9)	CWE-732	It works on every bucket, database and server in the account, not just the app bucket.
9	No `Condition`	CWE-732	No limit on prefix, IP address, VPC or MFA, so a stolen key can be used from anywhere.

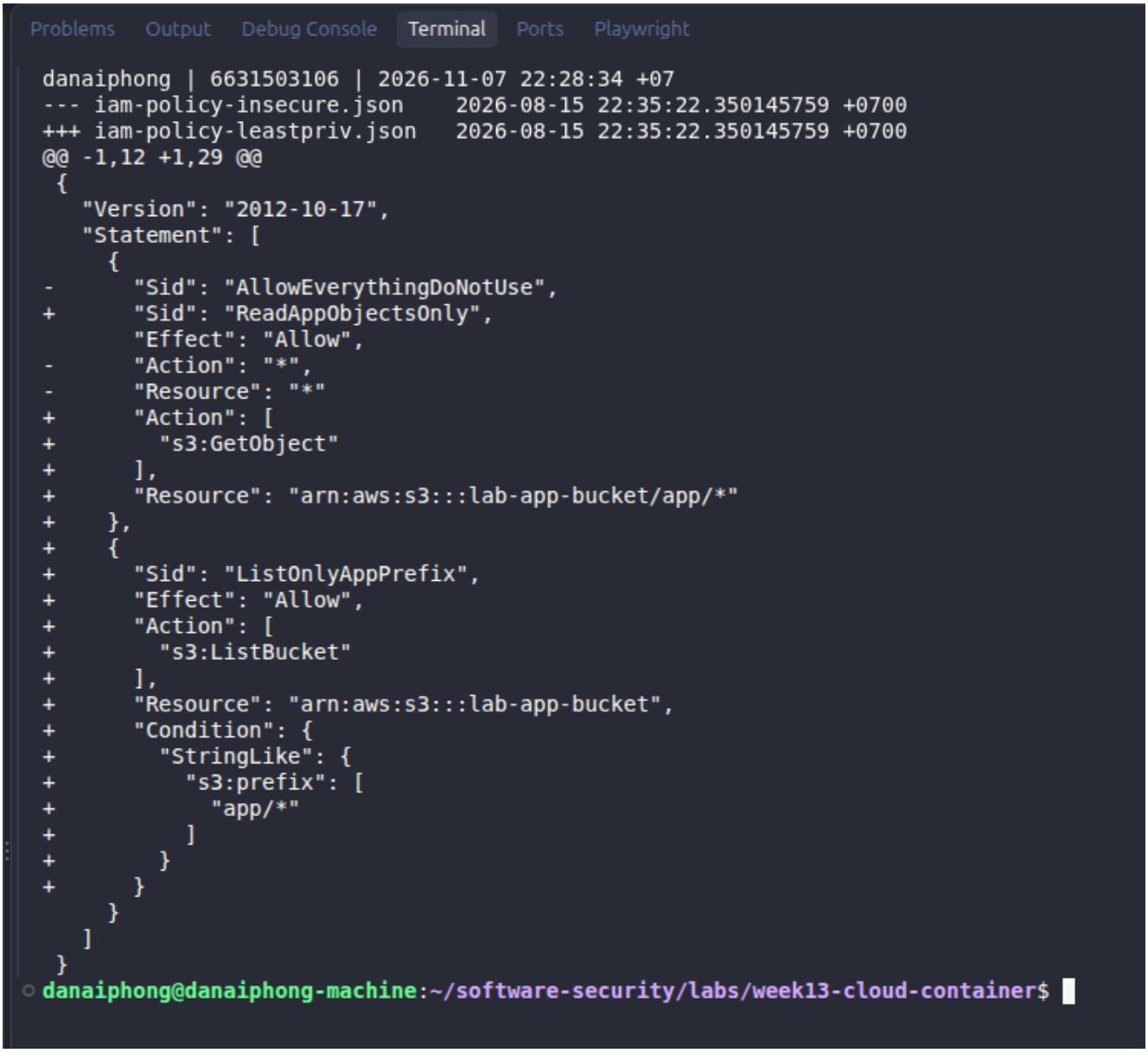
Blast radius: if these keys leak, the attacker gets admin on the whole AWS account. They could download or delete all S3 data, run crypto-mining servers on our bill, and create their own IAM users so they still have access after we change the leaked key.

Command for the comparison (saved in `evidence/iam-diff.txt`):

```
diff -u iam-policy-insecure.json iam-policy-leastpriv.json
```

Task 2 screenshot:

Task 2 - diff of the insecure and least-privilege IAM policies



• **Task 3 - Secrets & storage (25 min).** *Goal:* show secrets must not live in the image. *Steps:* explain how `ENV API_TOKEN` / `AWS_SECRET_ACCESS_KEY` are exposed via `docker history`; propose where they belong instead (secrets manager / mounted secret / orchestrator secret per `harden.md`). *Deliverable:* 2-3 sentence remediation + the `.dockerignore` entries you would add to stop `COPY . .` leaking `.git/.env` (CWE-538).

Task 3 - Answer

Commands (output in `evidence/history-insecure.txt` and `evidence/inspect-env-insecure.txt`):

```
$ docker build -f Dockerfile.insecure -t week13-insecure:lab .
$ docker history --no-trunc week13-insecure:lab | grep -oE 'ENV (API_TOKEN|AWS_SECRET_ACCESS_KEY)=[^ ]+'
ENV AWS_SECRET_ACCESS_KEY=AKIAEXAMPLEdeadbeefdeadbeef
ENV API_TOKEN=sk_live_supersecret_DO_NOT_DO_THIS

$ docker inspect --format '{{json .Config.Env}}' week13-insecure:lab
[..., "API_TOKEN=sk_live_supersecret_DO_NOT_DO_THIS",
"AWS_SECRET_ACCESS_KEY=AKIAEXAMPLEdeadbeefdeadbeef"]
```

Both secrets are shown in plain text. Docker keeps every ENV line in the image metadata, so a person who pulls the image can read them without running the container. `trivy image` also found the secrets in the image config and in two copied files (`/app/Dockerfile.insecure` and

`/app/evidence/scan-before.txt`), which got in because of `COPY . .` (see `evidence/image-insecure.txt`).

Remediation: I would remove the secrets from the image and give them to the container only when it runs. The options are a secrets manager (AWS Secrets Manager or Vault), a Docker/Kubernetes secret mounted as a file such as `/run/secrets/api_token`, or for AWS an IAM role on the task or pod so we do not need a long-lived key at all. The leaked values should also be rotated, since older images in the registry still contain them.

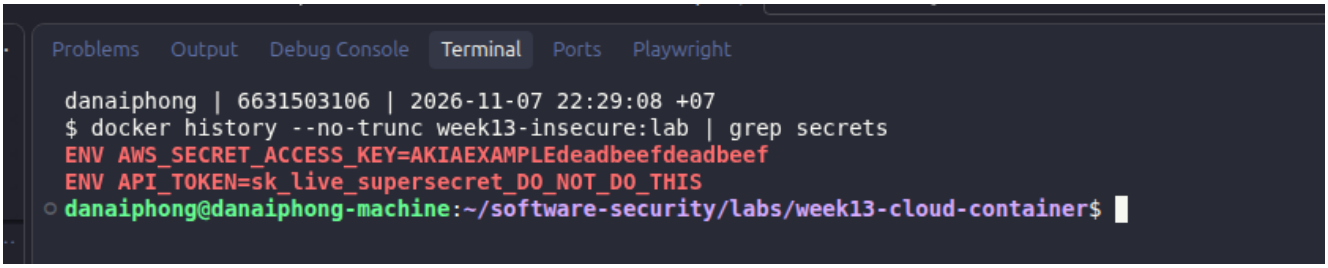
`.dockerignore` I would add:

```
.git
.env
*.env
keys/
*.pem
*.key
evidence/
Dockerfile*
*.md
```

I tested it on a copy of the lab where I added a fake `.env`, a fake `keys/id_rsa` and a `.git` folder. After building `Dockerfile.insecure` again, `/app` only had `app.py`, `requirements.txt`, the IAM JSON files, `img` and `scan.sh`, and `.git`, `.env` and `keys/` were not there (`evidence/dockerignore-check.txt`). I still think `.dockerignore` is only a backup. It is better to write `COPY app.py .` like the hardened file does.

Task 3 screenshot:

Task 3 - docker history --no-trunc week13-insecure:lab showing both baked secrets



• **Task 4 - Hardened-Dockerfile + least-priv-IAM defense (50 min).** *Goal:* prove the "after" is clean. *Steps:* (a) re-run `bash scan.sh` (or `trivy config`) against `Dockerfile.hardened` and capture the **reduced** finding count (Trivy's config scanner does not parse standalone IAM JSON, so confirm the IAM fix manually per Task 2 by diffing `iam-policy-leastpriv.json` against the insecure policy); (b) for the Dockerfile, point to the fix for each Task-1 defect (digest pin, multi-stage venv, distroless runtime, `USER 65532:65532`, no secrets in ENV, copy only `app.py`); (c) for IAM, confirm the rewrite scopes `s3:GetObject` to `arn:aws:s3:::lab-app-bucket/app/*` and adds the `s3:prefix` `Condition`; (d) list the extra runtime flags from `harden.md` (`--read-only`, `--cap-drop ALL`, `--security-opt no-new-privileges`). *Deliverable:* before/after finding counts, the defect→fix mapping, and the diffed policy/Dockerfile with one sentence per fix.

Task 4 - Answer

(a) Before and after

Check	Insecure	Hardened
`trivy config` misconfigurations (MEDIUM and up)	4 (1 MEDIUM, 1 HIGH, 2 CRITICAL)	0
`trivy image` OS CVEs (HIGH, CRITICAL)	282 (267 HIGH, 15 CRITICAL), Debian 13.7	69 (67 HIGH, 2 CRITICAL), Debian 12.13
`trivy image` Python package CVEs (HIGH and up)	5	2 (from packages inside setuptools in the venv)
Secrets found by `trivy image`	4 (2 in image config, 2 in copied files)	0
Image size	1.65 GB	125 MB
User when running	`uid=0(root)`	`uid 65532 gid 65532`

The config findings went from 4 to 0 (`evidence/scan-after.txt`). But I want to point out that the hardened image still has CVEs. Its distroless base is pinned to an older digest, and Trivy still finds 69 HIGH/CRITICAL OS CVEs in it. Pinning makes the build repeatable, but somebody still needs to update the digest and scan again from time to time.

(b) Defect to fix in `Dockerfile.hardened` (full diff in `evidence/dockerfile-diff.txt`)

#	Defect	Fix in hardened file	Why it helps
1	`FROM python:latest`	Line 15 `python:3.11-slim@sha256:cdbd05fb...` and line 42 `distroless/python3-debian12@sha256:2fdb0540...`	The base image only changes when someone edits the digest (CWE-1104).
2	Secrets in `ENV`	Lines 54-55 only set `PATH` and `PYTHONPATH`	`docker inspect` does not show any credential now (CWE-798/200).
3	`COPY . .`	Line 20 `COPY requirements.txt .`, line 28 `COPY app.py .`, and line 48 copies only `/app/app.py` to the final stage	I checked and `/app` contains only `['app.py']`, so `.git` or `.env` cannot leak (CWE-538).
4	`chmod -R 777`	Removed, files keep default root-owned permissions	The app user (65532) cannot change its own code (CWE-732).
5	Runs as root	Line 59 `USER 65532:65532`	If there is an RCE, the attacker is a normal user, not root (CWE-250).
6	Unpinned `pip install`	Multi-stage build, `pip install --no-cache-dir` into `/venv` in a build stage that is not shipped	Build tools are not in the final image. But `requirements.txt` still says `Flask>=3.1.3`, so to fully fix it I would use `Flask==3.1.3` with `--require-hashes`.
-	Big attack surface	Distroless runtime (line 42)	No shell and no package manager, sh could not start in the container.

(c) IAM rewrite (`evidence/iam-diff.txt`)

```
-      "Sid": "AllowEverythingDoNotUse",
```

```
-       "Action": "*",
-       "Resource": "*"
+       "Sid": "ReadAppObjectsOnly",
+       "Action": ["s3:GetObject"],
+       "Resource": "arn:aws:s3:::lab-app-bucket/app/*"
+     },
+     {
+       "Sid": "ListOnlyAppPrefix",
+       "Action": ["s3:ListBucket"],
+       "Resource": "arn:aws:s3:::lab-app-bucket",
+       "Condition": { "StringLike": { "s3:prefix": ["app/*"] } }
```

• `Action: "*"` became only `s3:GetObject` and `s3:ListBucket`. The app can read files, but it cannot write, delete, or do anything with IAM (CWE-269).

• `Resource: "*"` became `arn:aws:s3:::lab-app-bucket/app/*` for reading and the bucket ARN for listing (CWE-732).

• A `Condition` with `s3:prefix = app/*` was added, so the app can only list its own folder in the bucket.

The policy matches what the task asks for: `s3:GetObject` is limited to `arn:aws:s3:::lab-app-bucket/app/*` and there is an `s3:prefix` condition. I only reviewed the JSON. I did not test it on a real AWS account.

(d) Runtime flags (from `harden.md`, tested with the hardened image, `evidence/runtime-flags.txt`)

```
$ docker run -d --rm --read-only --cap-drop ALL --security-opt no-new-privileges \
    --memory 128m --pids-limit 64 -p 127.0.0.1:5013:5000 week13-hardened:lab
$ curl -s http://127.0.0.1:5013/health
{"status":"healthy"}
ReadonlyRootfs=true CapDrop=[ALL] SecurityOpt=[no-new-privileges] User=65532:65532
$ docker exec <id> python3 -c 'open("/app/x","w")'
OSError: [Errno 30] Read-only file system: '/app/x'
```

• `--read-only`: nothing can be written to the container filesystem, so an attacker cannot drop files or change the code.

• `--cap-drop ALL`: removes Linux capabilities like `NET_RAW` and `CHOWN`, which are often used in escape attacks.

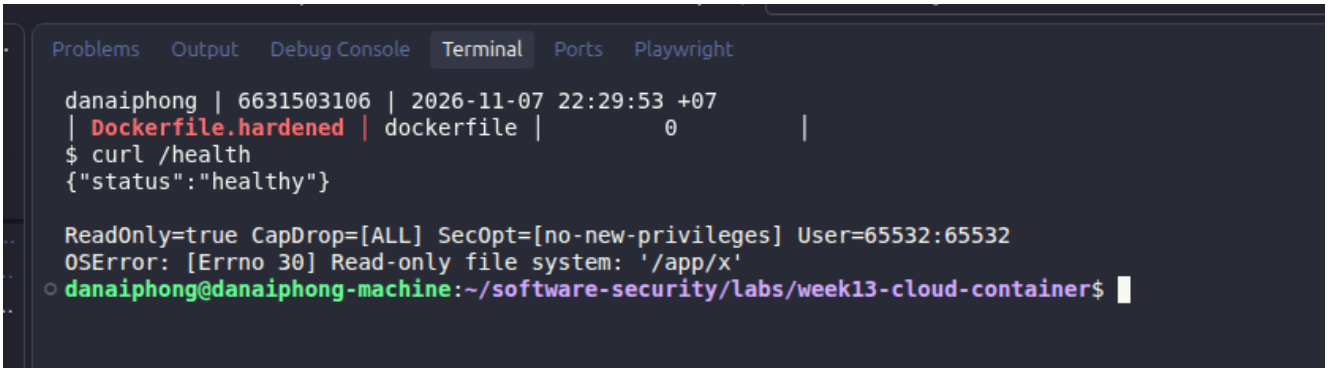
• `--security-opt no-new-privileges`: setuid programs cannot give more privileges.

• `--memory` and `--pids-limit`: limit DoS and fork bombs. Also, never run with `--privileged` or mount `/var/run/docker.sock`.

The app still answered `/health` normally with all of these flags on.

Task 4 screenshot:

Task 4 - hardened scan result (0 misconfigurations) and the container running with runtime flags



• **Task 5 - Flag tally (20 min).** *Goal:* score the hunt. *Steps:* one flag per misconfig found **and** fixed (6 container + 3 IAM = 9). *Deliverable:* flag count + the single biggest-risk misconfig and why.

Task 5 - Answer

Category	Found and fixed	Flags
Container (`:latest`, secret in ENV, `COPY . .`, `chmod 777`, root, unpinned pip)	6 / 6	6
IAM (`Action:*`, `Resource:*`, no `Condition`)	3 / 3	3
Total		9 / 9

Defect 6 is only partly fixed in the given hardened file because `requirements.txt` still uses >=. I still counted it, because I wrote the full fix (exact version and hashes) in Task 4.

Biggest risk: I think it is the `AWS_SECRET_ACCESS_KEY` inside the image, together with the `Action:*`/`Resource:*` policy. Anyone can read the key with `docker history`, and if that key has the admin policy, the attacker controls the whole AWS account and not only this container. The other problems make an attack easier, but these two together are already a full account takeover.

Part 4 - Reflection

1. **Mapping.** For three findings, write the line of the artifact → CWE → OWASP 2025 A02, and the one-line fix.

Answer:

Artifact line	CWE	OWASP 2025	One-line fix
`Dockerfile.insecure:16 ENV API_TOKEN=sk_live_...`	CWE-798	A02 Security Misconfiguration	Delete it and mount the token at runtime from a secrets manager.
`Dockerfile.insecure` (no USER line)	CWE-250	A02 Security Misconfiguration	Add `USER 65532:65532` and use a distroless base.
`iam-policy-insecure.json:8-9 "Action": "*", "Resource": "*"`	CWE-269 / CWE-732	A02 Security Misconfiguration	Allow only `s3:GetObject` on `arn:aws:s3:::lab-app-bucket/app/*`.

2. **Real incident.** Briefly describe a real breach caused by a cloud/container misconfiguration (e.g., a public S3 bucket leak, or an over-permissive IAM role). What single control from this lab would have prevented it?

Answer: I chose the Capital One breach from 2019. The attacker used an SSRF bug in a badly configured web application firewall to get the temporary credentials of the IAM role on that EC2 server. The role had much more S3 access than a firewall needs, so the attacker was able to list and copy data from more than 700 buckets, which had information of about 100 million customers. Least-privilege IAM from Task 2 and Task 4 would have prevented most of this. A firewall role does not need to read customer data, so with a limited policy the stolen credentials would not have been very useful.

3. **Best mitigation.** Of everything you did, which control gives the most risk reduction per unit of effort, and why? Argue for least-privilege IAM vs. distroless+non-root.

Answer: In my opinion least-privilege IAM gives the most benefit for the effort. The new policy is around 20 lines of JSON, and it reduces the damage no matter how the key leaks, whether from the image, an SSRF to the metadata service, a stolen laptop or a log file. Distroless and non-root were also easy here (a different `FROM` and one `USER` line), but they only help after the attacker is already inside this container, and they do not help if the key leaks in another way. So I would fix IAM first. I would still do distroless and non-root too, because in my test they did not break the app at all.

Grading rubric (100)

Component	Points	What earns full marks
Part 2 - Lecture questions	20	All 5 answered correctly with CWE/A02 reasoning
Part 3 - Tasks + evidence	40	Tasks 0-5 complete; commands, outputs, and screenshots present
Defense (Task 4 hardening)	25	Hardened Dockerfile + least-priv IAM shown to reduce findings; per-fix justification
Part 4 - Reflection	15	Accurate mapping, relevant incident, well-argued best mitigation

Evidence & Integrity (required)

• **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** - a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

• **Personalized flag (if this lab issues one):** This lab does not give a personalized flag. The "flags" for this week are the 9 misconfigurations counted in Task 5. *Flags are unique per student - submitting another student's flag is a violation. How to submit: **learn.zcr.ai/submit** (full guide: `SUBMISSION.md` in the repo root).*

• **Explain in your own words** *(graded on your reasoning, not copied text):* 1. What did you do, and **why did the vulnerability work**? 2. **Why does your fix actually stop it** - and what could still break it?

Evidence explanation

1. I scanned both Dockerfiles with Trivy, built the insecure image, and got the API token and AWS key out of it with `docker history` and `docker inspect`. This worked because Docker saves every `ENV` line in the image metadata. The container also ran as root with a `777 /app` folder, so nothing inside it was protected. The IAM policy was a problem by itself, because with * on * any leaked key is an admin key.

2. The hardened image has no secrets in it, runs as uid 65532 on distroless without a shell, and with `--read-only` it cannot write to its own code folder. With the new IAM policy, a leaked key can only read `lab-app-bucket/app/*`. Some things can still go wrong: the pinned distroless base still has 69 HIGH/CRITICAL CVEs until the digest is updated, `requirements.txt` still uses >=, a secret mounted at runtime can still be read if someone gets RCE in the container, and I did not test the IAM policy on real AWS.

Audit the AI (required)

AI is a power tool you must **distrust** - you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

2. **Find what's wrong or risky** in it - insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

3. Produce the **correct, verified** version yourself and explain in 2-3 sentences why the AI's output was insufficient.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Audit answer

Prompt: "Fix the security problems in this Dockerfile" (with `Dockerfile.insecure` pasted).

AI answer (full):

Here is a more secure version. I pinned the base image to a slim tag, moved the secret to a build argument so it is no longer hard-coded, added `--no-cache-dir`, and switched to a non-root user.

```
FROM python:3.12-slim
ARG API_TOKEN
ENV API_TOKEN=$API_TOKEN
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
USER nobody
EXPOSE 5000
CMD ["python", "app.py"]
```

Build it with `docker build --build-arg API_TOKEN=your_token -t app .` so the token is not stored in the Dockerfile.

Problems I found:

• `ARG API_TOKEN` + `ENV API_TOKEN=$API_TOKEN`: the token is not written in the Dockerfile anymore, but it is still saved in the image. I built it with `--build-arg API_TOKEN=sk_test_from_build_arg`, and `docker inspect` showed `"API_TOKEN=sk_test_from_build_arg"`. `docker history --no-trunc` showed it too. Trivy still gave 2 CRITICAL DS-0031 findings, one "in ARG" and one "in ENV" (`evidence/ai-fix-check.txt`). The AI says the secret is fixed, but it is still there.

• `FROM python:3.12-slim` is still a tag and not a digest, so it can change. It only passes DS-0001 because the tag is not `latest`.

• `COPY . .` was not changed, so `.git`, `.env` and keys would still be copied (CWE-538). There is also no `.dockerignore`.

• `USER nobody` is OK, but there is no multi-stage build or distroless image, so pip, the shell and build tools are still in the final image.

• It does not say that the leaked token has to be rotated.

Correct version I checked: I used `Dockerfile.hardened` (digest pins, multi-stage, distroless, `COPY app.py`, `USER 65532:65532`, no secret in ARG or ENV) together with the `.dockerignore` from Task 3, and passed the secret only at runtime, for example:

```
docker run --read-only --cap-drop ALL --security-opt no-new-privileges \
  -v "$PWD/api_token:/run/secrets/api_token:ro" week13-hardened:lab
```

If a secret is needed during the build, for example for a private package index, BuildKit `RUN --mount=type=secret,id=...` can be used because it does not save the secret in a layer. I

checked that the hardened file has 0 Trivy config findings and that `docker inspect` shows no secret in its env. The AI's answer was not enough because it only moved the secret from the Dockerfile to the build command. The image still has the secret inside, so it is still CWE-798.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2-3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* - explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification - this trains problem decomposition and AI literacy (Denny et al. 2024).*

A. Plain-English answer

The insecure Dockerfile puts a small Flask app into an image together with an API token and an AWS key. It also copies every file in the folder, lets anyone change the code, and runs as root. Anyone who downloads the image can read the keys from the image history, and if someone gets code running in the container they are already root. The IAM policy makes it worse because the key is allowed to do anything in the AWS account.

B. Final prompt and verification

Final prompt:

"Rewrite this Dockerfile for a Flask app (`app.py`, `requirements.txt`). Requirements: (1) pin every `FROM` to an `@sha256:` digest, not only a tag. (2) Use a multi-stage build and a distroless Python runtime with the same Python version as the build stage. (3) Copy only `app.py` and `requirements.txt`, and give me a `.dockerignore` that excludes `.git`, `.env`, keys and `*.pem`. (4) Do not put any secret in `ENV` or `ARG`. Explain how to pass `API_TOKEN` at runtime as a mounted file. (5) Run as a numeric non-root user. (6) No `chmod 777`. After the Dockerfile, list the commands I should run to check it: `trivy config`, `docker inspect` for env, and `docker history --no-trunc | grep -i token`."

My first prompt gave the `ARG`/`ENV` answer above, so I added point (4). I added "numeric" in (5) and "same Python version" in (2) because a venv made with another Python version does not run on distroless.

Result: the answer had the same structure as `Dockerfile.hardened`. I ran the commands from the prompt: `trivy config` gave 0 misconfigurations for the hardened file, `docker inspect` only showed `PATH`, `SSL_CERT_FILE`, `LANG` and `PYTHONPATH` with `User=65532:65532`, and `docker history --no-trunc` did not show any token. Reading the secret with `docker history` does not work anymore because the secret is not in the image.

Screenshot checklist

Before each screenshot, run this in the same terminal so the stamp is in the picture:

```
printf '%s | %s | ' "$(whoami)" '6631503106'; TZ=Asia/Bangkok date '+%F %T %Z'
```

File	What to show
`img/task0.png`	`docker version` + `bash scan.sh` report summary (insecure 4, hardened 0)
`img/task1.png`	The DS-0001, DS-0002 and DS-0031 findings from the scan
`img/task2.png`	`diff -u iam-policy-insecure.json iam-policy-leastpriv.json`
`img/task3.png`	`docker history --no-trunc week13-insecure:lab | grep -E 'API_TOKEN|AWS_SECRET'`
`img/task4.png`	Hardened scan (0) + container running with `--read-only --cap-drop ALL --security-opt no-new-privileges` and `curl /health`

References

1. OWASP Docker Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

2. Trivy Dockerfile checks DS-0001, DS-0002, DS-0031: https://avd.aquasec.com/misconfig/ds-0001 (and `/ds-0002`, `/ds-0031`)

3. AWS shared responsibility model: https://aws.amazon.com/compliance/shared-responsibility-model/

4. AWS IAM best practices (least privilege): https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

5. Docker build secrets (`RUN --mount=type=secret`): https://docs.docker.com/build/building/secrets/

6. Capital One 2019 incident: U.S. Department of Justice press release, July 29, 2019.

My local results are saved in `evidence/` (scan-before, scan-after, image-insecure, history-insecure, inspect-env-*, insecure-runtime, hardened-runtime, runtime-flags, iam-diff, dockerfile-diff, dockerignore-check, ai-fix-check).